Change request log

1. Concept Location

Step #	Description	Rationale
1	We ran the system	We used Visual code IDE to run the code. Used the search tool in IDE after performing ant fullDeploy and ant reload commands.
2	We interacted with the system: after logging in we entered the schedule screen.	To acquaint ourselves with certain system features like where the password change functionality is located, how a new user information is saved.
3	Our initial approach was to search for "user" within the codebase using Visual Studio Code's regular expression search functionality.	This broad search yielded numerous results, making it difficult to pinpoint the specific file relevant to password modifications. We needed a more targeted approach to identify the right code segment.
4	Attempting to locate password-related code, we searched for "NewPassword" using the IDE's regular expression search functionality.	While this led us to files likely containing password data, it didn't provide the specific code needed for updating passwords. Our search needed further refinement.
5	Expanding our search, we focused on "admin" reasoning that only an administrator could change passwords.	This search led us to potential files like user.java and permission.java, assuming they might contain code related to administrator privileges.
6	Within the identified files, we specifically searched for the "isAdmin" method to pinpoint password-related logic.	This search revealed relevant methods like isAdmin() and setAdmin() in userDao.java. Notably, the frequent presence of "admin" and "user" terms in this file further suggested its connection to password management.
7	To gain a comprehensive understanding of user-related functionalities, we examined the entirety of User.java and UserDao.java, carefully reviewing included methods.	Recognizing the need to modify passwords based on our task, we identified the updateUser() method as the most relevant for making the necessary changes

Time spent (in minutes): 70 minutes
Classes and methods inspected:

- \mangoSource\src\com\serotonin\mango\vo\User.java
  - isAdmin(), setAdmin()
- \mangoSource\src\com\serotonin\mango\db\dao\UserDao.java
  - updateUser()

2. Impact Analysis

Step #	Description	Rationale
1	We made a list of methods called by Impact classes	To track the classes that could be impacted by the change.

2	We identified the problem by observing errors when attempting to change an administrator's password, resulting in an error popup.	This initial observation prompted the need for further investigation into the root cause of the issue
3	We reviewed the updateUser method within the UserDao class and found that it needed modifications to facilitate smooth password updates for administrators.	Since, the UserDao class manages database operations for user management in the Mango M2M application, handles CRUD operations, user data mapping and permission management , we thought of modifying the code in this class.
4	Analyzing the components, we focused primarily on the UserDao class to determine potential impacts on related modules	By narrowing down the scope to the UserDao class, we aimed to efficiently assess the extent of changes required.
5	Assessing dependencies allowed us to understand potential ripple effects of the change across the system.	This step ensured that we considered all interconnected components that might be affected by the proposed modifications.

Time spent (in minutes): 40 minutes

- \mangoSource\src\com\serotonin\mango\db\dao\UserDao.java
  - UserDao class
  - insertUser -method
- \mangoSource\src\com\serotonin\mango\db\dao\UserDao.java
  - UserDao class
  - updateUser -method

- \mangoSource\src\com\serotonin\mango\db\dao\UserDao.java
  - UserDao class
  - getUser -method

3. Actualization

Step #	Description	Rationale
1	Identified an issue with potential NullPointerExceptions occurring in the updateUser method within the UserDao class, especially when handling null values for user attributes.	The presence of NullPointerExceptions posed a risk to the stability and reliability of the user update functionality, necessitating a solution to handle such scenarios.
2	Modified the updateUser method to include exception handling using a try-catch block, encapsulating the database update operation.	Exception handling was introduced to gracefully capture and manage any potential runtime exceptions that might occur during the database update process, preventing unexpected crashes and providing better error visibility and control.
3	Implemented checks to ensure non-null values for user attributes such as username, password, email, phone, and homeUrl before invoking the database update operation.	By validating and assigning default empty string values to potentially null attributes, we aimed to mitigate the risk of NullPointerExceptions and ensure the consistency of data passed to the database update statement.

4	Ensured that the boolToChar method is correctly invoked to convert boolean values, such as user.isAdmin() and user.isDisabled(), to their corresponding database representation.	This step guarantees the proper handling of boolean attributes during the database update process, maintaining data integrity and consistency in the underlying database tables.
5	Updated the exception handling mechanism to log any encountered exceptions and provide informative error messages to aid in debugging and troubleshooting.	Logging exceptions and throwing a custom runtime exception with detailed error messages enhances the maintainability and supportability of the codebase, enabling developers to diagnose and address issues more effectively

Time spent (in minutes): 60 minutes

Inspected

- \mangoSource\src\com\serotonin\mango\db\dao\UserDao.java
  - UserDao class
  - insertUser -method
- \mangoSource\src\com\serotonin\mango\db\dao\UserDao.java
  - UserDao class
  - updateUser -method
  - isAdmin, setAdmin, isDisabled methods

- \mangoSource\src\com\serotonin\mango\db\dao\UserDao.java
  - UserDao class
  - getUser -method

Changed

- \mangoSource\src\com\serotonin\mango\db\dao\UserDao.java
  - UserDao class
  - updateUser -method

4. Postfactoring

Step #	Description	Rationale
1	After the logging changes, we executed the unit tests for the UserDao class to ensure that the introduction of logging did not impact the existing functionality. Additionally, we executed the system tests to verify that logging statements are generated as expected.	This step ensures that the refactoring did not introduce regressions and that the logging is functioning correctly.
2	We reviewed the exception handling strategy in the updateUser method. Instead of catching a general Exception, we refined the catch block to specifically catch DataAccessException (assuming it's the exception type thrown by the ejt.update method) and rethrow it as a more specific custom exception (e.g., UserDaoException).	This provides more precise information about the type of exception that occurred, making it easier to diagnose and handle specific issues.
3	We added appropriate Javadoc comments to the updateUser method, explaining its purpose, parameters, and potential exceptions.	This improves code documentation and makes it easier for other developers to understand the intended use of the method.

Time Spent : 15 minutes

5. Validation

Step #	Description	Rationale	Result
1	Unit testing: Update a user with all valid data	Ensure individual method correctness	Pass
2	Regression testing: Update a user with all valid data	Ensure existing functionality remains unaffected	Pass
3	System testing: Execute a complete user update operation	Ensure the system functions as expected as a whole	Pass
4	Execute a user update with large password and small password	Assess system performance with large data inputs	Pass
5	Attempt to update a non-existing user		
6	Update a user password with various data types	Test system's robustness against unexpected data types	Pass
7	Integration testing: Integrate with logging and error handling	Test integration with logging and error-handling mechanisms	Pass
8	Integrate with the database and update a user	Ensure seamless integration with the database	Pass
9	Acceptance testing: Update a user through the application UI	Verify user update functionality through the UI and check whether all requirements are satisfied	Pass

Time spent (in minutes): 30 min

6. Summary of the change request

Phase	Time (minutes)	No. of classes inspected	No. of classes changed	No. of methods inspected	No. of methods changes
Concept location	70	2	2	3	1
Impact Analysis	40	1	1	3	1
Actualization	60	1	1	3	1
Post-factoring	15	2	0	3	0
Validation	30	2	0	3	0
Total	215 min	2	4	3	3

7. Conclusions

The change request for the **UserDao** class presented several significant challenges throughout the modification process. Initially, understanding the intricacies of the existing codebase and the interactions between various methods was crucial for devising appropriate modifications. Identifying the potential impact of the requested changes on other components of the system required meticulous analysis to prevent unintended consequences. Implementing robust exception handling mechanisms and ensuring proper null value handling in database operations were critical challenges that demanded careful consideration and testing to maintain application stability. Additionally, managing transactions effectively to preserve data integrity during database operations posed a significant challenge, necessitating thorough implementation and testing. Despite these challenges, a systematic approach, thorough analysis, and diligent testing were instrumental in successfully addressing the change request while upholding the reliability and functionality of the system.